

CURSO

Desarrollo de Aplicaciones con IA

Prompts Prácticos para Claude Code

Multi-Agent Workflows · TalentBridge CV Evaluator

Generado el 12 de May de 2026

8

Prompts
Prácticos

3

Fases de
Implementación

4

Agentes
Especializados

6

Slash
Commands

Tabla de Contenidos

01	Orquestador de Análisis del Prototipo Análisis técnico, arquitectura y propuestas de mejora con agentes paralelos
02	Fase 1 — Reorganización Frontend / Backend Separación de carpetas, setup de Express, package.json raíz con concurrently
03	Fase 2 — Migración de Gemini al Backend Endpoint /api/evaluate, validaciones, limpieza del frontend
04	Fase 3 — Proxy, Health Check y Documentación Vite proxy, banner de estado del servidor, README actualizado
05	Orquestador de Implementación Completa Las 3 fases coordinadas con gates de verificación entre fases
06	Configuración Claude Code (Rules, Skills, Agents, Commands) CLAUDE.md con rules, 6 slash commands, 4 agentes especializados, skills README
07	Documento de Arquitectura General a Particular 5 capas: Negocio, Funcional, Integración, Técnica, Roadmap
08	Skill dev-server — Subir y Bajar el Stack Slash command /dev-server up down status con manejo de PIDs

PROMPT 01

Orquestador de Análisis del Prototipo

MULTI-AGENTE · ANÁLISIS

Objetivo: Lanzar 3 agentes especializados en paralelo para auditar el prototipo de Google AI Studio. Cada agente produce un artefacto Markdown en /docs/. Al terminar, el orquestador sintetiza un plan de acción unificado.

CONTEXTO GLOBAL

```
# Lee estos archivos antes de lanzar agentes:
# - src/App.tsx (625 líneas – toda la UI y lógica de estado)
# - src/services/geminiService.ts (3 llamadas Gemini con structured output)
# - src/types.ts (Criterion, CandidateRanking, ExecutiveSummary, AppStep)
# - vite.config.ts / package.json (Tailwind v4, motion/react, lucide-react)
```

AGENTE 1 — ANALYST · Auditor técnico de código fuente

Deliverable: /docs/01-analisis-tecnico.md

Secciones requeridas:

- Flujo de Datos completo (JD → Criterion[] → CandidateRanking[] → ExecutiveSummary)
- Componentes y Responsabilidades
- Deuda Técnica (código duplicado, tipos débiles, manejo de errores)
- Riesgos de Seguridad (exposición GEMINI_API_KEY en bundle cliente)
- Limitaciones del Modelo Actual (3 prompts Gemini: estructura, schema, limitaciones)

AGENTE 2 — ARCHITECT · Documentador de arquitectura y UX

Deliverable: /docs/02-arquitectura-y-ux.md

Secciones requeridas:

- Diagrama Mermaid (flowchart TD) con steps del wizard, llamadas Gemini, estado
- Tipos y Contratos — descripción de cada campo en types.ts
- Análisis UX por Paso — fricciones identificadas en cada step
- Puntos de Fricción (near-ties, validación pesos 100%, separador ---, etc.)
- Propuestas de Mejora UX (mínimo 5 propuestas concretas)

AGENTE 3 — IMPROVER · Estratega de mejoras técnicas

Deliverable: /docs/03-propuestas-de-mejora.md

Secciones requeridas:

- Mejoras a los 3 prompts Gemini (schemas más ricos, reintentos con backoff)

- Propuesta de refactoring de App.tsx (hooks: useEvaluation, useCriteria, useCandidates)
- Backlog Funcional Priorizado (5 mejoras con impacto/esfuerzo S/M/L)
- Roadmap Sugerido en 3 fases

INSTRUCCIONES DE ORQUESTACIÓN

1. Crea el directorio /docs/ si no existe.
2. Lanza los 3 agentes EN PARALELO con la herramienta Task.
3. Espera a que los 3 terminen.
4. Lee los 3 documentos generados.
5. Escribe /docs/00-plan-de-accion.md con:
 - Resumen ejecutivo (máx. 200 palabras)
 - Top 5 acciones ordenadas por impacto
 - Dependencias entre mejoras
 - Estimación total de esfuerzo por fase
6. Confirma que los 4 documentos están listos con sus rutas.

PROMPT 02

Fase 1 — Reorganización Frontend / Backend

IMPLEMENTACIÓN · FASE 1

Objetivo: Separar el proyecto monolítico en carpetas frontend/ y backend/. Crear el servidor Express con el endpoint de health check. Configurar scripts concurrentes en la raíz. No mover lógica de Gemini en este paso.

CONTEXTO

El código actual está en la raíz. src/App.tsx llama a Gemini directamente con @google/genai. La clave está en .env.local como VITE_GEMINI_API_KEY — el prefijo VITE_ la expone al navegador. **Eso es inseguro.**

PASO 1 Mover el proyecto a frontend/

- Mueve src/, public/, index.html, vite.config.ts, tsconfig.json a frontend/
- Copia package.json a frontend/package.json
- Elimina dependencias de servidor (express, cors) del frontend/package.json
- Scripts frontend: dev (puerto 5173), build, preview, lint

PASO 2 Crear backend/ desde cero

- package.json con: express, @google/genai, cors, dotenv
- devDependencies: typescript, tsx, @types/express, @types/cors, @types/node
- tsconfig.json: outDir dist/, strict true
- src/index.ts: Express en puerto 3001, middleware cors + json (límite 5mb)
- GET /api/health → { status: 'ok', version: '1.0' }
- .env con GEMINI_API_KEY= (sin prefijo VITE_)
- .gitignore: node_modules/, dist/, .env

PASO 3 Configurar package.json raíz con concurrently

- scripts.dev: corre frontend y backend en paralelo
- scripts.dev:frontend y dev:backend por separado
- scripts.install:all: instala en raíz, frontend y backend

VERIFICACIÓN

```
npm run install:all
npm run dev:backend &
curl -s http://localhost:3001/api/health
# Debe responder: {"status":"ok","version":"1.0"}
```

PROMPT 03

Fase 2 — Migración de Gemini al Backend

IMPLEMENTACIÓN · FASE 2

Objetivo: Mover toda la lógica de Gemini del frontend al backend. Crear el endpoint POST /api/evaluate con validaciones. El frontend pasa a consumir la API interna vía fetch. Eliminar @google/genai del bundle del cliente.

BACKEND — backend/src/routes/evaluate.ts

Endpoint: POST /api/evaluate

Body recibido:

```
{ jobDescription: string,  
  cvs: Array<{ filename: string, content: string }> }
```

Validaciones (400 con mensaje en español):

- **jobDescription < 10 palabras** → "Descripción de cargo muy corta. Incluye al menos 10 palabras."
- **cvs vacío o más de 20** → "Cantidad de CVs fuera de rango. Máximo 20 CVs por evaluación."
- **!process.env.GEMINI_API_KEY** → responde 500: "Servidor mal configurado — contacta al administrador."

■ ■ Envuelve todo en try/catch. Loggea errores al servidor con console.error, pero NUNCA expongas el stack trace ni la clave al cliente.

FRONTEND — Limpieza de frontend/

- Elimina el import de @google/genai en frontend/src/App.tsx
- Reemplaza la llamada a Gemini por fetch('/api/evaluate', { method: 'POST', body: JSON.stringify(...) })
- Elimina VITE_GEMINI_API_KEY de frontend/.env.example y .env.local
- Ejecuta: cd frontend && npm uninstall @google/genai

VERIFICACIÓN

```
# 1. La clave NO debe aparecer en el frontend  
grep -r 'VITE_GEMINI_API_KEY' frontend/ # debe retornar vacío  
  
# 2. @google/genai NO debe estar en frontend/package.json  
cat frontend/package.json | grep genai # debe retornar vacío  
  
# 3. TypeScript sin errores  
cd frontend && npm run lint
```

PROMPT 04

Fase 3 — Proxy, Health Check y Documentación

IMPLEMENTACIÓN · FASE 3

Objetivo: Configurar el proxy de Vite para que el frontend hable con el backend sin CORS en desarrollo. Agregar un indicador visual de estado del servidor. Actualizar el README con la nueva arquitectura.

PASO 1 — Proxy en frontend/vite.config.ts

```
server: {
  port: 5173,
  host: '0.0.0.0',
  proxy: {
    '/api': {
      target: 'http://localhost:3001',
      changeOrigin: true
    }
  }
}
```

PASO 2 — Health Check UI en frontend/src/App.tsx

- Estado: type ServerStatus = 'checking' | 'ok' | 'error'
- useEffect al montar: llama GET /api/health con AbortController (timeout 3s)
- Si responde OK → status 'ok', luego setTimeout 2000ms → 'checking' (desaparece)
- Si falla → status 'error'
- Banner rojo (error): '■■ Servidor no disponible. Ejecuta npm run dev desde la raíz.'
- Indicador verde (ok): '● Servidor conectado' — desaparece a los 2 segundos
- Usa estado React — NO usar alert()

PASO 3 — README.md actualizado

- Estructura de carpetas del proyecto
- Instrucciones de instalación: npm run install:all
- Dónde va la clave de Gemini: backend/.env (NO en frontend)
- Cómo correr en dev: npm run dev desde la raíz
- Tabla de puertos: frontend 5173 / backend 3001
- Sección de seguridad: por qué la clave no va en el cliente

VERIFICACIÓN E2E

```
# Inicia todo
npm run dev &

# Verifica health directo y proxied
curl -s http://localhost:3001/api/health
curl -s http://localhost:5173/api/health # vía proxy Vite

# CRÍTICO: la clave NO debe aparecer en el bundle del cliente
grep -r 'GEMINI_API_KEY' frontend/dist/ 2>/dev/null # debe estar vacío
# Si la encuentras: REVIERTE y arregla antes de continuar
```

PROMPT 05

Orquestador de Implementación Completa

MULTI-AGENTE · IMPLEMENTACIÓN

Objetivo: Coordinar 3 fases de implementación. Cada fase tiene agentes paralelos con propietario exclusivo de archivos. Las fases son secuenciales con gates de verificación entre ellas. El orquestador NO avanza si una fase falla.

FASE 1 Reorganización de carpetas

F1-A Frontend Mover — Archivos: *frontend/*

Mueve *src/*, *index.html*, *vite.config.ts*, *tsconfig.json*. Adapta *package.json*.

F1-B Backend Creator — Archivos: *backend/ (nueva)*

Crea Express + health endpoint + *tsconfig* + *.env* files.

F1-C Root Config — Archivos: *package.json raíz*

Scripts con concurrently: *dev*, *dev:frontend*, *dev:backend*, *install:all*.

Gate de verificación:

```
npm run install:all && curl http://localhost:3001/api/health
```

FASE 2 Mover lógica Gemini al backend

F2-A Gemini Backend — Archivos: *backend/src/routes/*

Crea *evaluate.ts* con *POST /api/evaluate* + validaciones + manejo errores.

F2-B Frontend Cleaner — Archivos: *frontend/src/*, *frontend/package.json*

Elimina *@google/genai*, reemplaza llamada por *fetch*, limpia *.env*.

Gate de verificación:

```
grep -r 'VITE_GEMINI' frontend/ (vacío) && cd frontend && npm run lint
```

FASE 3 Proxy, Health Check UI y Documentación

F3-A Proxy Config — Archivos: *frontend/vite.config.ts*

Agrega *server.proxy* para */api* → *localhost:3001*.

F3-B Health UI — Archivos: *frontend/src/App.tsx*

Banner error/ok con *useEffect* + *AbortController*.

F3-C README Updater — Archivos: *README.md raíz*

Documenta estructura, instalación, puertos, seguridad.

Gate de verificación:

```
curl http://localhost:5173/api/health && grep -r 'GEMINI_API_KEY' frontend/dist/ (vacío)
```

Regla cardinal: Si el gate de cualquier fase falla, el orquestador se detiene, lee los logs de error, corrige el problema y solo entonces avanza a la siguiente fase. Nunca heredes un estado roto a la siguiente fase.

PROMPT 06

Configuración Claude Code: Rules, Skills, Agents, Commands

CONFIGURACIÓN · CLAUDE CODE

Objetivo: Crear toda la configuración nativa de Claude Code para convertir el repositorio en un entorno de desarrollo asistido por IA especializado en TalentBridge. Lee los 4 documentos de /docs/ antes de empezar.

ARTEFACTO 1 — CLAUDE.md: Rules por dominio

- R-SEC

Seguridad de credenciales

NUNCA escribas GEMINI_API_KEY en frontend/. NUNCA uses VITE_ para secrets externos. Verifica con: `grep -r 'GEMINI_API_KEY' frontend/` (debe retornar vacío).
- R-API

Contrato del endpoint /api/evaluate

Errores siempre en español como { error: string }. Códigos: 400=input inválido, 500=fallo servidor, 200=éxito. Nunca incluyas stack traces al cliente.
- R-GEMINI

Llamadas al modelo

Siempre `responseMimeType: 'application/json'` + `responseSchema`. Siempre `withRetry()` (máx 3, `backoff 800ms`). Modelo: `gemini-2.5-flash-preview-05-20`.
- R-TYPES

Tipado TypeScript

Prohibido `any`. Usa `unknown` + `type guards`. Tipos compartidos en `shared/types.ts`. Toda función pública con tipos explícitos.
- R-STATE

Gestión de estado React

Estado en custom hooks, no en App.tsx. Componentes de paso son funciones puras. Prohibido `alert()` y `console.log` en UI.
- R-GIT

Workflow de commits

Formato: `type(scope): descripción en inglés`. Tipos: `feat`, `fix`, `refactor`, `chore`, `docs`, `test`.

ARTEFACTO 2 — .claude/commands/ (6 Slash Commands)

Comando	Descripción
<code>/add-endpoint</code>	Scaffolding de nuevo endpoint Express con validaciones, try/catch y tipos
<code>/check-security</code>	Auditoría: busca API keys expuestas, VITE_ secrets, archivos .env trackeados
<code>/add-criterion</code>	Agrega campo al schema de evaluación (types.ts + responseSchema + UI)
<code>/extract-component</code>	Extrae componente inline de App.tsx a su propio archivo tipado
<code>/verify-stack</code>	Inicia backend + frontend, verifica health en ambos puertos, mata procesos

`/export-csv`

Implementa exportación de rankings a CSV con Blob + createObjectURL

ARTEFACTO 3 — .claude/agents/ (4 Agentes Especializados)

Agente	Modelo	Cuándo usarlo
security-auditor	opus	Audita exposición de API keys. Úsalo antes de cada deploy.
gemini-optimizer	opus	Optimiza prompts y responseSchema. Muestra antes/después con comentarios.
refactor-guide	sonnet	Guía la extracción de hooks y componentes. Verifica que App.tsx queda con menos líneas.
ux-reviewer	sonnet	Evalúa claridad, feedback, recuperabilidad y near-ties del wizard.

PROMPT 07

Documento de Arquitectura — General a Particular

DOCUMENTACIÓN · 5 CAPAS

Objetivo: Generar docs/ARQUITECTURA.md como historia de arriba hacia abajo: primero el problema de negocio, luego cómo se resuelve funcionalmente, cómo se integra, cómo está construido, y hacia dónde va. Cualquier persona — técnica o no — debe poder leerlo.

CAPA 1 VISIÓN DE NEGOCIO

- 1.1 Propósito del producto (qué problema, para quién, por qué IA generativa)
- 1.2 Usuarios y roles — tabla: Rol | Necesidad | Interacción
- 1.3 Proceso de negocio — diagrama Mermaid flowchart LR (antes vs. después de TalentBridge)
- 1.4 Métricas de éxito (KPIs: tiempo preselección, consistencia, cobertura)
- 1.5 Restricciones y supuestos (qué NO hace el sistema)

CAPA 2 ARQUITECTURA FUNCIONAL

- 2.1 Mapa de capacidades — mindmap: Ingesta, Análisis IA, Presentación, Exportación
- 2.2 Flujo de usuario — sequenceDiagram: Reclutador → Frontend → Backend → Gemini
- 2.3 Casos de uso críticos (evaluar CVs, ajustar pesos, exportar ranking)
- 2.4 Reglas de negocio (pesos 100%, min 10 palabras, max 20 CVs, near-tie ≤ 0.5)
- 2.5 Modelo de información — erDiagram: JD, Criterion, CV, CandidateRanking, ExecutiveSummary

CAPA 3 ARQUITECTURA DE INTEGRACIÓN

- 3.1 Diagrama de contexto C4 (TalentBridge + Reclutador + Gemini API + Browser)
- 3.2 Contratos de API interna — GET /api/health y POST /api/evaluate con tipos TS
- 3.3 Integración Gemini — tabla de las 3 llamadas (función, modelo, schema, timeout)
- 3.4 Variables de entorno — tabla con seguridad: qué NUNCA va en el cliente
- 3.5 Proxy Vite dev vs. reverse proxy en producción

CAPA 4 ARQUITECTURA TÉCNICA

- 4.1 Estructura del repositorio (árbol con descripción por carpeta)
- 4.2 Stack tecnológico — tabla: Capa | Tecnología | Versión | Justificación
- 4.3 Diagrama de componentes React — subgraph ACTUAL vs. OBJETIVO
- 4.4 Diagrama de módulos backend (index → routes → validators → Gemini SDK)
- 4.5 Modelo de datos técnico — interfaces TS completas con descripción de campos
- 4.6 ADRs simplificados — 5 decisiones clave con alternativas y consecuencias
- 4.7 Deuda técnica registrada — tabla con ID, impacto, esfuerzo, sprint objetivo
- 4.8 Seguridad — amenazas, controles implementados vs. pendientes

CAPA 5 ROADMAP TÉCNICO

- 5.1 Estado actual vs. objetivo (tabla por dimensión: Arquitectura, Seguridad, Testing...)
- 5.2 Gantt Mermaid de las 3 fases del plan de acción
- 5.3 Riesgos técnicos — tabla: riesgo, probabilidad, impacto, mitigación

INSTRUCCIONES DE FORMATO

- Todo el documento en español
- Cada sección comienza con 1-2 líneas de contexto (por qué importa esa sección)
- Los diagramas Mermaid van en bloques ````mermaid ... ````
- Máximo 3 niveles de encabezado (`##`, `###`, `####`)
- Al final: sección `##` Glosario con términos de negocio y técnicos
- Verificar al terminar: `wc -l docs/ARQUITECTURA.md`

PROMPT 08

Skill dev-server — Subir y Bajar el Stack

SLASH COMMAND · GESTIÓN DE PROCESOS

Objetivo: Crear `.claude/commands/dev-server.md` con un slash command que gestione el ciclo de vida del stack de desarrollo. Los PIDs se guardan en `/tmp/talentbridge-pids.txt` para poder matar cada proceso individualmente.

Uso: `/dev-server [up | down | status]`

Acción: UP

- Verifica si `/tmp/talentbridge-pids.txt` existe → si hay PIDs, avisa y detente
- Inicia backend: `npm run dev:backend > /tmp/talentbridge-backend.log 2>&1 &`
- Guarda PID: `echo $! > /tmp/talentbridge-pids.txt`
- Espera 3s → verifica: `curl -s http://localhost:3001/api/health`
- Si backend falla → muestra `tail -20` del log y detente (no inicies frontend)
- Inicia frontend: `npm run dev:frontend > /tmp/talentbridge-frontend.log 2>&1 &`
- Agrega PID: `echo $! >> /tmp/talentbridge-pids.txt`
- Espera 5s → verifica: `curl` retorna HTTP 200 en `localhost:5173`
- Muestra tabla de URLs: Backend 3001, Frontend 5173, Health check

Acción: DOWN

- Lee PIDs de `/tmp/talentbridge-pids.txt`
- Mata cada PID con `kill $pid 2>/dev/null`
- Limpieza defensiva: `kill -f 'vite' && kill -f 'tsx watch'`
- Verifica puertos libres: `lsof -ti:3001,5173` (si retorna algo, lo mata)
- Elimina `/tmp/talentbridge-pids.txt`
- Confirma: Puerto 3001 liberado / Puerto 5173 liberado

Acción: STATUS

- `curl -s --max-time 2 http://localhost:3001/api/health`
- `curl -s --max-time 2 -o /dev/null -w '%{http_code}' http://localhost:5173`
- `cat /tmp/talentbridge-pids.txt` (PIDs registrados)
- Reporte: Backend `■/■` | Frontend `■/■` | PIDs: [lista]

Nota importante: Usa `npm run dev:backend` y `dev:frontend` por separado (NO `npm run dev` con `concurrently`) para obtener PIDs individuales y poder matar cada proceso de forma independiente.

VERIFICACIÓN FINAL

```
# Confirma que el archivo fue creado  
cat .claude/commands/dev-server.md | head -5  
# Debe mostrar: # dev-server
```

Resumen de Aprendizajes Clave

- 1 Los agentes paralelos ahorran tiempo**
Cuando las tareas no tienen dependencias entre sí (analizar código, documentar arquitectura, proponer mejoras), lanzarlas en paralelo reduce el tiempo total significativamente.

- 2 Las fases secuenciales necesitan gates**
Nunca heredes un estado roto. Verifica cada fase con comandos concretos (curl, grep, lint) antes de que el siguiente agente empiece a construir sobre ese estado.

- 3 El propietario de archivo previene conflictos**
Asignar a cada agente un conjunto exclusivo de archivos elimina los merge conflicts y hace el trabajo paralelo predecible y reproducible.

- 4 La seguridad es una constraint del prompt**
Especificar explícitamente 'NUNCA expongas la API key' y 'verifica con grep' convierte la seguridad en un criterio de aceptación, no en un afterthought.

- 5 Los slash commands son memoria del equipo**
Codificar los patrones repetidos (add-endpoint, check-security, extract-component) en commands evita que cada desarrollador reinvente el flujo cada vez.

- 6 La documentación de 5 capas facilita onboarding**
Un documento que va de negocio a código permite que un PM, un nuevo dev o un stakeholder lean solo las capas que les interesan sin perderse en detalles técnicos prematuros.

TalentBridge CV Evaluator · Prompts Prácticos
Curso Desarrollo de Aplicaciones con IA · 2026